Bhavya Marripudi

Rochester hills, MI | bhavya1102m@gmail.com | GitHub | portfolio | 940-843-7625

Professional Summary

Detail-oriented Android Developer with 4.5+ years of experience specializing in Kotlin, Jetpack Compose, MVVM, and Firebase. Adept at building scalable, modern Android applications using clean architecture and integrating backend services such as AWS and Firebase. Experienced with connected-vehicle data (CAN) and smart-glasses / XR displays (Viture, Xreal, Magic Leap). Proven track record of delivering production-ready apps, debugging performance bottlenecks, and automating CI/CD pipelines. Experienced in agile environments, collaborating with cross-functional teams, and driving full SDLC from planning to deployment.

Skills

- **Languages:** Kotlin, Java, Dart, XML, SQL, Python, JSON
- **Android:** Jetpack Compose, ViewModel, LiveData, Room, DataStore, Navigation Component, WorkManager, Paging 3, BroadcastReceiver, AlarmManager
- **Architecture & Patterns:** MVVM, Clean Architecture, StateFlow, Kotlin Coroutines, Repository Pattern
- **Smart Glasses & XR:** Viture Luma Ultra, Xreal, Magic Leap, OpenGL ES (3D panels), Android Canvas/Bitmap rendering, secondary-display (Presentation) output, HUD design for see-through displays
- **Vehicle Data:** CAN bus signal decoding, connected-vehicle telemetry, GPS / compass sensor fusion
- **Cloud:** Firebase (Auth, Firestore, Realtime DB, Crashlytics, Analytics, Security Rules), AWS
- **DevOps & CI/CD:** GitHub Actions, Bitrise, Jenkins, ProGuard Rules, Build Variants, APK Signing, Release Build Configuration
- **Databases, Testing & Cross-Platform:** SQLite, Room, Firestore | Espresso, JUnit, Mockito | Flutter (Beginner)
- **Security & Performance:** Encrypted Shared Preferences, Performance Optimization (Lazy Loading, Caching, Coroutines)
- **APIs:** Valhalla API, Overpass API, Miovision API, OpenWeather API

GitHub Projects

SafetyApp – SafteyApp
- Built a safety-focused emergency alert app using Kotlin, Firebase, and Google Maps SDK.
- Enabled real-time location tracking and emergency alert broadcasting to selected contacts.
- Used MVVM architecture and Firebase Realtime Database for scalable updates.

AI ChatBot – AI_ChatBot
- Created a conversational AI chatbot using OpenAI API and Jetpack Compose for UI.
- Handled asynchronous networking with Kotlin coroutines for smooth interactions.
- Designed reusable UI components for quick layout scaling.

CustomWidgets – CustomWidgets
- Developed a set of reusable Jetpack Compose UI widgets to streamline interface consistency.
- Modularized design system and improved UI build speed in multiple Android apps.

SmartNotesApp – SmartNotesApp
- Developed an advanced Android notes app with reminders, snooze notifications, calendar/date-based filtering, and export to TXT/PDF, using **Jetpack Compose**, **Room Database**, **Hilt**, **MVVM**, and **Material 3** for a modern, maintainable architecture.
- Implemented search, filtering, alarm notifications, dynamic theming, and PDF export with **ViewModel**, **StateFlow**, **BroadcastReceiver**, and **Kotlin Coroutines**, ensuring smooth UX and scalable codebase.

Multilingual News Aggregator with AI Summarizer – MultilingualNewsAggregatorWithAISummarizer
- Built a modular Android app using Kotlin, Jetpack Compose, and Hilt to display global news with real-time language toggle and AI-based summarization.
- Implemented trending carousel, article sharing, and dark/light theme using MVVM and StateFlow for scalable architecture.

Professional Experience

Android Developer – Akkodis Group (Client: General Motors) | Aug 2025 - Present

Project: Advanced Navigation Based on Connected Data
- Migrated major modules from Java to Kotlin, enabling coroutine-based concurrency and improving maintainability.
- Performed code optimization and refactoring, reducing technical debt and improving runtime stability.
- Designed and implemented a CSV playback framework for vehicle telemetry (GPS, speed, heading, traffic signal timing) to simulate real-world driving scenarios offline.
- Integrated Valhalla API for advanced routing, map-matching, and navigation logic, along with trace generation and playback tools for scenario reproduction.
- Consumed and integrated data from Overpass API, Miovision feeds, and OpenWeather APIs, enriching navigation with road names, stop signs, traffic signals, and weather overlays.
- Built heading-aware road name resolution logic to reduce ambiguity between parallel/intersecting roads in navigation UI.
- Enhanced CHState orchestration to seamlessly switch between live telemetry and offline simulation without conflicts.
- Implemented user settings features including CSV file picker, flexible playback modes, and visualization controls.
- Collaborated with GM engineers to validate connected vehicle data services and ensure reliability of navigation features across test environments.
- Designed and implemented a Foreground Service–based WebSocket server (Android 14 compliant) to support real-time communication between vehicle systems and smart glasses devices.
- Architected background service lifecycle management with Android 12+/14 FGS permission handling and dataSync service type compliance.
- Built UDP-based peer discovery module to enable automatic smart glasses device detection across local networks.
- Engineered socket-based communication layer with dynamic port management and network resilience handling.
- Ensured Android OS compatibility by implementing Foreground Service permissions (FOREGROUND_SERVICE, FOREGROUND_SERVICE_DATA_SYNC) for SDK 34+.
- Optimized service startup logic from Application lifecycle to meet modern Android background execution restrictions.
- Developed robust logging and diagnostics framework to trace connection states, discovery issues, and network instability.
- Coordinated integration testing across connected device environments to validate end-to-end telemetry streaming reliability.
- Built the **lane-level speed data** pipeline: polled a backend feed for per-lane speeds by road segment, parsed the segment/lane JSON, and supplemented it with predicted per-lane speeds (CSV) on I-696 segments.
- Implemented **lane-level map matching** with JTS lane polygons and PROJ coordinate transforms to identify the host vehicle's lane (1–4) from GPS and direction of travel, with a nearest-lane fallback.

- Designed a **lane-change recommendation algorithm** that compares the next-segment speed of the current lane against the other lanes and advises LEFT / RIGHT when the relative gain exceeds 15% (with speed sanity limits), returning stay / change / invalid decisions.
- Added **lane-change detection** with debouncing and range validation, publishing direction and time to shared state for the HUD overlay.

Smart Glasses Application – Advanced HUD & Camera Integration

- Architected and developed a real-time Android HUD application for smart glasses, integrating vehicle telemetry, traffic events, lane guidance, and weather overlays using Kotlin and MVVM architecture.
- Designed a WebSocket-based communication layer to stream live connected vehicle data (speed limits, stop signs, lane changes, traffic signals) with low-latency rendering.
- Implemented a wireless RTSP camera integration module using Media3 ExoPlayer to support baby-seat monitoring and future rear backup camera mode.
- Engineered subnet auto-discovery and IP persistence logic for wireless camera detection, including concurrent scanning control and connection stabilization.
- Built gesture-based UI control (double-tap toggle) to dynamically switch between navigation HUD and external camera preview modes.
- Optimized real-time rendering pipelines to ensure smooth overlay updates while maintaining 60 FPS camera preview performance.
- Implemented fail-safe handling for RTSP buffering, reconnection logic, and state-based playback monitoring using ExoPlayer listeners.
- Developed heading-aware lane visualization and corner detection logic to improve driver situational awareness.
- Integrated battery status, Wi-Fi health monitoring, and system diagnostics within the HUD environment.
- Applied concurrency best practices (thread pooling, race-condition prevention, lifecycle-aware operations) to eliminate unstable scan and connection behavior.

Viture, Xreal & Magic Leap Smart Glasses – Near-Vehicle and Driving HUD

- Built an Android host app that renders the in-car HUD and Near-Vehicle panels onto **Viture Luma Ultra** and **Xreal** see-through glasses through the phone/tablet's secondary display, sharing one vehicle-data module with the existing Magic Leap build.
- Developed an **OpenGL ES 3D panel renderer** that places 512×512 canvas textures in space and locks them to the car's heading, so panels stay fixed relative to the vehicle as the driver turns their head.
- Designed an animated **Near-Vehicle radar UI** for see-through displays (light on black, outlines over fills, minimal text): sonar rings, car bearing marker, distance counter, and a walking figure driven by real GPS movement, replacing a text-only screen with icon-based status chips (CAN link, lock, fuel, window).
- Decoded vehicle **CAN bus** signals (lock state, windows, turn signals, speed, shift lever, and fuel level from Fuel_Information 0x10800040) and added link-loss detection so the UI refreshes on disconnect and keeps the **last known** vehicle status, persisted with SharedPreferences. Added a window-open alert (red pulsing icon, edge glow, voice prompt) for a locked car or lost link.
- Extended the Near-Vehicle screen to **Magic Leap** as well, reusing the shared vehicle state and fuel-gauge renderer across Magic Leap, Viture, and Xreal.
- Mapped steering-wheel cruise-switch CAN frames to LEFT / RIGHT / SELECT so the driver navigates glasses panels hands-free; added automatic **Near-Vehicle ↔ Driving** mode switching from the transmission shift lever with manual override.
- Built the driving HUD canvas (turn indicators, speed limit, current speed, road name) and optimized the per-frame renderer by reusing bitmaps, canvases, and paints to avoid allocation churn on the render loop.

Android Developer – People Tech Group (Client: General Motors) | Dec 2019 – June 2023

- Developed the **Vehicle Info Android module** for GM's connected car system, enabling users to view detailed real-time vehicle data.
- Integrated **Retrofit** to consume VIN-based APIs for fetching diagnostics, mileage, engine status, and fuel levels.
- Followed **MVVM architecture** and utilized **Jetpack components** (LiveData, ViewModel, Navigation) to ensure a scalable and maintainable codebase. Integrated **Google Maps SDK** to visualize historical route data with markers, routes, and geofencing alerts.
- **Improved app boot time by 35%** by optimizing coroutine usage, caching strategies, and lazy data loading.
- Maintained **OTA (Over-the-Air)** update functionality, ensuring seamless firmware syncing between mobile and embedded vehicle systems.
- Implemented **custom logging and crash reporting** using Firebase Crashlytics and integrated analytics to monitor app performance across devices.
- Collaborated with cross-functional teams (backend, QA, design) in the U.S. using **Agile/Scrum**, participating in sprint planning and daily stand-ups.
- Conducted **code reviews** and enforced Kotlin best practices across modules to reduce technical debt.
- Built **unit and instrumentation tests** using JUnit, Mockito, and Espresso, improving test coverage by over 45%.
- Contributed to the migration of legacy Java modules to modern **Kotlin-based architecture**.
- Documented technical workflows, API contracts, and setup guides for internal developer onboarding and handoff.

Education

University of North Texas — MS in Computer Science | Denton, TX | May 2025
Jawaharlal Nehru Technological University — B.Tech in CSE | India | 2019

Certifications

Intro to Android Mobile App Development (Meta) | Java for Android (Vanderbilt) | API's (Meta) – all on Coursera